

Имплементация на модула „Паспорти“

I. Подготовка на маршрутите и страниците на модула

Първо нека дефинираме маршрутите за този нов модул, за целта променете „routes.ts“ файла по следния начин:

```
// Users
...prefix('users', [index('routes/users/users.tsx')]),

// Passports
...prefix('passports', [
  index('routes/passports/passports.tsx'),
  route('/:passportId', 'routes/passports/passport-details.tsx'),
  route('/create', 'routes/passports/passport-create.tsx'),
  route('/:passportId/edit', 'routes/passports/passport-
edit.tsx'),
]),
```

В модул потребители оставаме само index страницата, която разработихме в миналото упражнение и дефинираме нови маршрути за новия модул „Паспорти“. Маршрутите са както следва: index страница, страница за визуализация на един паспорт, създаване на паспорт и редактиране на съществуващ паспорт.

Нека създадем базови файлове за тези маршрути. В директорията „routes“, създайте нова директория „passports“ за съхранението на тези нови страници. В нея директория създайте файла „passports.tsx“ и поставете следния код:

```
import type { Route } from './+types/passports'

export function meta({}: Route.MetaArgs) {
  return [
    { title: 'React Router App' },
    { name: 'description', content: 'Паспорти' },
  ]
}

export async function clientLoader({ request }:
Route.ClientLoaderArgs) {}
```

```
export function HydrateFallback() {
  return null
}

export default function Passports() {
  return null
}
```

След това създайте файла „passport-details.tsx“ и поставете следния код:

```
import type { Route } from '../types/passport-details'

export function meta({}: Route.MetaArgs) {
  return [
    { title: 'React Router App' },
    { name: 'description', content: 'Паспорти' },
  ]
}

export async function clientLoader({
  params: { passportId },
}: Route.ClientLoaderArgs) {}

export function HydrateFallback() {
  return null
}

export default function PassportDetailsPage() {
  return null
}
```

След това създайте файла „passport-create.tsx“ и поставете следния код:

```
import type { Route } from '../types/passport-create'

export function meta({}: Route.MetaArgs) {
  return [
    { title: 'React Router App' },
    { name: 'description', content: 'Създаване на паспорт' },
  ]
}

export default function PassportCreatePage() {
  return null
}
```

И накрая създайте файла „passport-edit.tsx“ и поставете следния код:

```
import type { Route } from './+types/passport-edit'

export function meta({}: Route.MetaArgs) {
  return [
    { title: 'React Router App' },
    { name: 'description', content: 'Редактиране на паспорт' },
  ]
}

export async function clientLoader({
  params: { passportId },
}: Route.ClientLoaderArgs) {}

export function HydrateFallback() {
  return null
}

export default function PassportEditPage() {
  return null
}
```

II. Имплементация на index страницата

По подобен начин на index страницата на модул „Потребители“, добавете зареждането на данните във функцията „clientLoader“, като не забравяте да промените ендпоинта на „passports“.

След това добавете компонента, който ще се споделя между компонента на страницата и компонента, който се визуализира докато се зарежда.

```
type PassportsPageProps = {
  children?: ReactNode
}

function PassportsPage({ children }: PassportsPageProps) {
  return (
    <>
      <div className='flex items-center justify-between mb-4'>
        <h1 className='text-2xl font-bold'>Паспорти</h1>
        <Button
          asChild
          className='bg-green-600 hover:bg-green-700 dark:bg-green-600 dark:hover:bg-green-700'
          >
          <Link to='/passports/create'>
```

```

        <PlusIcon />
        Създай паспорт
      </Link>
    </Button>
  </div>
  {children}
</>
)
}

```

В този компонент имаме освен заглавието на модула и бутон за създаване на нов паспорт, който препраща потребителя към страницата с формата за създаване на паспорт.

Накрая променете функцията с компонента на страницата и този за зареждане по следния начин:

```

export function HydrateFallback() {
  return (
    <PassportsPage>
      <PassportsIndexTableSkeleton />
    </PassportsPage>
  )
}

export default function Passports() {
  const loaderData = useLoaderData<typeof clientLoader>()

  return (
    <PassportsPage>
      <PassportsIndexTable data={loaderData} />
    </PassportsPage>
  )
}

```

Следва да създадем тези компоненти „PassportsIndexTable“ и „PassportsIndexTableSkeleton“. За целта създайте нова директория „passports“ в директорията „app“ и в нея създайте файла „passports-index-table.tsx“ и поставете следния код:

```

type PassportsIndexTableProps = {
  data: PassportsIndexResponse
}

export function PassportsIndexTable({ data }:
PassportsIndexTableProps) {
  const columns: PaginatedTableColumn<Passport>[] = useMemo(

```

```

    () => [
      {
        key: 'name',
        header: 'Име',
      },
      {
        key: 'model',
        header: 'Модел',
      },
      {
        key: 'serialPrefix',
        header: 'Сериен префикс',
      },
      {
        key: 'fromSerialNumber',
        header: 'От сериен номер',
      },
      {
        key: 'toSerialNumber',
        header: 'До сериен номер',
      },
      {
        key: 'warrantyMonths',
        header: 'Гаранция (месеци)',
      },
    ],
    []
  )

  return <PaginatedTable data={data} columns={columns} />
}

export function PassportsIndexTableSkeleton() {
  return <PaginatedTableSkeleton columns={7} rows={10} />
}

```

За да може потребителя, когато натисне върху даден паспорт, да бъде препратен към страницата с визуализацията на неговите данни ще променим малко компонента „PaginatedTable“. Добавете към неговите „props“ функцията „getRowLink“ по следния начин:

```

type PaginatedTableProps<T> = {
  ...
  getRowLink?: (item: T) => string
}

export function PaginatedTable<T extends Record<string, any>>({

```

```

...
  getRowLink,
}: PaginatedTableProps<T>) {
  ...
}

```

След това променете начина, по който се рендерират редовете в таблицата:

```

{filteredItems.length > 0 &&
  filteredItems.map((item, rowIndex) => {
    const rowLink = getRowLink?.(item)

    return (
      <TableRow
        key={rowIndex}
        className={rowLink ? 'cursor-pointer' : undefined}
        onClick={rowLink ? () => navigate(rowLink) : undefined}
      >
        {columns.map((column, colIndex) => (
          <TableCell key={colIndex}>
            {column.cell
              ? column.cell(item)
              : String(item[column.key] ?? '')}
          </TableCell>
        ))}
      </TableRow>
    )
  })}
}

```

По този начин за всеки един ред, ако има функция „getRowLink“ подадена като проп, тя ще бъде извикана, като ще ѝ се подаде самият елемент, който се рендерира в момента. След като функцията върне линк към дадения елемент, при натискане на реда, потребителя ще бъде препратен към него линк.

Дефинирайте функцията „getRowLink“ в компонента „PassportsIndexTable“ и я подайте към „PaginatedTable“ по следния начин:

```

export function PassportsIndexTable({ data }:
PassportsIndexTableProps) {
  ...

  function getRowLink(passport: Passport) {
    return `/passports/${passport.id}`
  }

  return (

```

```

    <PaginatedTable data={data} columns={columns}
getRowLink={getRowLink} />
  )
}

```

Понеже паспортите не могат да бъдат филтрирани, както потребителите, ще трябва да показваме полето с търсачката опционално. За целта добавете „isSearchable“ към „props“ на „PaginatedTable“ по следния начин:

```

type PaginatedTableProps<T> = {
  ...
  isSearchable?: boolean
}

export function PaginatedTable<T extends Record<string, any>>({
  ...
  isSearchable = false,
}: PaginatedTableProps<T>) {
  ...
}

```

След което го използвайте при генерирането на списъка с филтрирани елементи „filteredItems“ по следния начин:

```

const filteredItems = useMemo(() => {
  if (!isSearchable || !searchQuery.trim()) {
    return data.items
  }

  const query = searchQuery.toLowerCase()
  return data.items.filter((item) => {
    return columns.some((column) => {
      if (column.searchable === false) return false

      const value = item[column.key]

      if (value == null) return false

      return String(value).toLowerCase().includes(query)
    })
  })
}, [data.items, searchQuery, columns, isSearchable])

```

И накрая показвайте опционално полето с търсачката по следния начин:

```
<div className='flex-1 max-w-xs'>
  {isSearchable && (
    <Input
      placeholder={searchPlaceholder}
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className='w-full'
    />
  )}
</div>
```

По този начин, само тогава когато се подава изрично флага „isSearchable“, ще се визуализира и ще работи търсачката. Затова трябва да го добавим към таблицата, която визуализира потребителите по следния начин:

```
export function UsersIndexTable({ data }: UsersIndexTableProps) {
  return <PaginatedTable data={data} columns={columns} isSearchable
/>
}
```

Когато „prop“ е булева стойност, то тогава може да се подаде без „={...}“. Това автоматично ще означава, че стойността му е „true“. Затова такава „prop“ се задават със стойности по подразбиране „false“ и само когато е необходимо се подава дадения „prop“, за да зададе стойност „true“.

Последното нещо, което трябва да добавим към тази таблица е колона с бутони за редакция и изтриване на паспорт. За целта ще добавим към типа „PaginatedTableColumn“ още един bug колона, която може да се дефинира.

```
export type PaginatedTableColumn<T> =
  | {
    key: keyof T
    header: string
    cell?: (item: T) => ReactNode
    searchable?: boolean
  }
  | {
    key: 'actions'
    header: string
    cell: (item: T) => ReactNode
    searchable?: never
  }
```


По този начин указваме, че могат да се дефинират колони, чиито „key“ са ключове от типа T и колона, чиито „key“ е „actions“. Забраняваме на колоната „actions“ да ѝ бъде дефинирано полето „searchable“, по което се филтрират елементите.

След това като последен елемент можем да дефинираме колоната „actions“:

```
{
  key: 'actions',
  header: 'Действия',
  cell: (passport) => (
    <div className='flex gap-1'>
      <Button
        size='icon-sm'
        variant='default'
        className='bg-blue-600 hover:bg-blue-700 text-white dark:bg-blue-600 dark:hover:bg-blue-700'
        asChild
        onClick={(e) => e.stopPropagation()}
      >
        <Link to={`/passports/${passport.id}/edit`} >
          <PencilIcon />
        </Link>
      </Button>
      <div onClick={(e) => e.stopPropagation()}>
        {/* Delete button */}
      </div>
    </div>
  ),
},
```

В тази колона дефинираме визуализацията на бутона за редакция, който ще препраща потребителя към формата за редактиране на дадения паспорт, както и ще дефинираме бутона за изтриване.

III. Страница за преглеждането на данните на един паспорт

Добавете функционалността за извличане на данните за един паспорт, като използвате неговото id, което получаваме от обекта „params“. Енпойнта за извличане на тези данни е „/passports/passportId“.

След това променете компонента на страницата и този за зареждане по следния начин:

```

export function HydrateFallback() {
  return <PassportDetailsCardSkeleton />
}

export default function PassportDetailsPage() {
  const loaderData = useLoaderData<typeof clientLoader>()

  return <PassportDetailsCard passport={loaderData} />
}

```

В директорията „app/passports“ създайте файла „passport-details-card.tsx“ и в него дефинирайте компонентите „PassportDetailsCard“ и „PassportDetailsCardSkeleton“:

```

export type PassportDetailsCardProps = {
  passport: Passport
}

export function PassportDetailsCard({ passport }:
PassportDetailsCardProps) {
  return (
    <Card>
      <CardHeader></CardHeader>
      <CardContent></CardContent>
    </Card>
  )
}

export function PassportDetailsCardSkeleton() {
  return (
    <Card>
      <CardHeader></CardHeader>
      <CardContent></CardContent>
    </Card>
  )
}

```

В компонента „PassportDetailsCard“ към „CardHeader“ добавете:

```

<div className='flex items-center justify-between'>
  <div>
    <h2 className='text-2xl font-bold'{passport.name}</h2>
    <p className='text-muted-foreground'>Модел: {passport.model}</p>
  </div>
  <div className='flex gap-2'>
    <Button
      variant='default'

```

```

        className='bg-blue-600 hover:bg-blue-700 text-white dark:bg-
blue-600 dark:hover:bg-blue-700'
        asChild
      >
      <Link to={`/passports/${passport.id}/edit`} >
        <PencilIcon />
        Редактирай
      </Link>
    </Button>
    {/* Delete button */}
  </div>
</div>

```

По този начин в „CardHeader“ ще визуализираме името на паспорта, както и модела. Ще се визуализират и бутоните за редакция и изтриване на този паспорт.

Към „CardContent“ добавяме:

```

<div className='grid grid-cols-1 md:grid-cols-2 gap-6'>
  <div className='space-y-4'>
    <div>
      <Label className='text-muted-foreground'>Име</Label>
      <p className='text-lg font-medium'>{passport.name}</p>
    </div>
    <div>
      <Label className='text-muted-foreground'>Модел</Label>
      <p className='text-lg font-medium'>{passport.model}</p>
    </div>
    <div>
      <Label className='text-muted-foreground'>Серийна
префикс</Label>
      <p className='text-lg font-medium'>{passport.serialPrefix}</p>
    </div>
  </div>
  <div className='space-y-4'>
    <div>
      <Label className='text-muted-foreground'>От серийен
номер</Label>
      <p className='text-lg font-
medium'>{passport.fromSerialNumber}</p>
    </div>
    <div>
      <Label className='text-muted-foreground'>До серийен
номер</Label>
      <p className='text-lg font-
medium'>{passport.toSerialNumber}</p>
    </div>
  </div>

```

```

    <div>
      <Label className='text-muted-foreground'>Гаранция
(месеци)</Label>
      <p className='text-lg font-
medium'>{passport.warrantyMonths}</p>
    </div>
  </div>
</div>

```

По този начин, в двете колони, ще се визуализират останалите данни на паспорта.

Съответно в компонента „PassportDetailsCardSkeleton“ към „CardHeader“ добавяме:

```

<div className='flex items-center justify-between'>
  <div className='space-y-2'>
    <Skeleton className='h-8 w-64' />
    <Skeleton className='h-5 w-48' />
  </div>
  <div className='flex gap-2'>
    <Skeleton className='h-10 w-20' />
    <Skeleton className='h-10 w-20' />
  </div>
</div>

```

И към „CardContent“:

```

<div className='grid grid-cols-1 md:grid-cols-2 gap-6'>
  <div className='space-y-4'>
    {Array.from({ length: 4 }).map((_, i) => (
      <div key={i} className='space-y-2'>
        <Skeleton className='h-4 w-32' />
        <Skeleton className='h-6 w-full' />
      </div>
    ))}
  </div>
  <div className='space-y-4'>
    {Array.from({ length: 3 }).map((_, i) => (
      <div key={i} className='space-y-2'>
        <Skeleton className='h-4 w-32' />
        <Skeleton className='h-6 w-full' />
      </div>
    ))}
  </div>
</div>

```

IV. Имплементиране на бутона за изтриване на паспорт

Когато потребителя се опита да изтрие паспорт, първо ще му визуализираме съобщение за потвърждение на това действие. За целта ще използваме компонента „AlertDialog“ от библиотеката „shadcn/ui“.

```
pnpm dlx shadcn@latest add alert-dialog
```

След това ще си създадем помощен компонент, който ще можем да го използваме в проекта, чрез който ще отваряме прозореца за потвърждение и ще обработваме отговора на потребителя. В директорията „app/components/core“ създайте файла „confirm-dialog.tsx“ и поставете следния код:

```
import type { VariantProps } from 'class-variance-authority'
import { useState } from 'react'
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from '~/components/ui/alert-dialog'
import { buttonVariants } from '~/components/ui/button'
import { Spinner } from '../ui/spinner'

export type ConfirmDialogProps = {
  title: string
  description: string
  onConfirm: () => void | Promise<void>
  children: (props: {
    onClick: () => void
    disabled: boolean
  }) => React.ReactNode
  cancelLabel?: string
  confirmLabel?: string
  variant?: VariantProps<typeof buttonVariants>['variant']
  disabled?: boolean
}
```

```

export function ConfirmDialog({
  title,
  description,
  onConfirm,
  children,
  confirmLabel = 'Потвърди',
  cancelLabel = 'Откажи',
  variant = 'destructive',
  disabled = false,
}: ConfirmDialogProps) {
  const [open, setOpen] = useState(false)
  const [isLoading, setIsLoading] = useState(false)

  const handleConfirm = async () => {
    setIsLoading(true)
    try {
      await onConfirm()
      setOpen(false)
    } catch (error) {
      console.error('Confirm action error:', error)
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <AlertDialog open={open} onOpenChange={setOpen}>
      {children({ onClick: () => setOpen(true), disabled })}
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>{title}</AlertDialogTitle>
          <AlertDialogDescription>{description}</AlertDialogDescripti
on>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel disabled={isLoading}>
            {cancelLabel}
          </AlertDialogCancel>
          <AlertDialogAction
            onClick={(e) => {
              e.preventDefault()
              handleConfirm()
            }}
            disabled={isLoading}
            variant={variant}
          >
            {isLoading && <Spinner />}
            {confirmLabel}
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>
  )
}

```

```

        </AlertDialogAction>
      </AlertDialogFooter>
    </AlertDialogContent>
  </AlertDialog>
)
}

```

По този начин дефинираме как ще изглеждат нашите съобщения за потвърждение на дадено действие, като можем да променяме заглавието, текста и текстовете на бутоните за „Отказ“ и „Потвърди“. Когато потребителят избере бутона „Потвърди“, се визуализира състояние на зареждане, с цел когато действието, което потвърждава потребителя изисква повече време.

Във файла „app/components/ui/alert-dialog.tsx“ променете компонента „AlertDialogAction“ по следния начин:

```

function AlertDialogAction({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Action> &
VariantProps<typeof buttonVariants>) {
  return (
    <AlertDialogPrimitive.Action
      className={cn(buttonVariants({ variant, size }), className)}
      {...props}
    />
  )
}

```

След като имаме помощния компонент „ConfirmDialog“, вече можем да имплементираме компонента с бутона за изтриване на един паспорт. За целта създайте файла „delete-passport-button.tsx“ в директорията „app/components/passports“ и поставете следния код:

```

import { useNavigate } from 'react-router'
import { Button, buttonVariants } from '~/components/ui/button'
import { TrashIcon } from 'lucide-react'
import useFetch from '~/lib/hooks/use-fetch.hook'
import { getApiUrl } from '~/lib/utils'
import { ConfirmDialog } from '~/components/core/confirm-dialog'
import type { Passport } from '~/models/passports.models'
import type { VariantProps } from 'class-variance-authority'

```

```

export type DeletePassportButtonProps = {
  passport: Passport
  size?: VariantProps<typeof buttonVariants>['size']
  showLabel?: boolean
  onDeleteSuccess?: () => void
}

export function DeletePassportButton({
  passport,
  size = 'default',
  showLabel = true,
  onDeleteSuccess,
}: DeletePassportButtonProps) {
  const navigate = useNavigate()
  const { fetch, isLoadingRef } = useFetch()

  const handleDelete = async () => {
    try {
      const apiUrl = getApiUrl(`/passports/${passport.id}`)
      await fetch(apiUrl, 'DELETE')

      if (onDeleteSuccess) {
        onDeleteSuccess()
      } else {
        navigate('/passports', { replace: true })
      }
    } catch (err) {
      console.error('Delete passport error:', err)
    }
  }

  return (
    <ConfirmDialog
      title='Изтриване на паспорт'
      description={`Сигурни ли сте, че искате да изтриете паспорт "${passport.name}"? Това действие не може да бъде отменено.`}
      confirmLabel='Изтрий'
      cancelLabel='Откажи'
      variant='destructive'
      onConfirm={handleDelete}
      disabled={isLoadingRef.current}
    >
      <Button
        size={size}
        variant='destructive'
        onClick={handleDelete}
        disabled={isLoadingRef.current}
      />
    </ConfirmDialog>
  )
}

```



```
        >  
        <TrashIcon />  
        {showLabel && 'Изтрий'}  
    </Button>  
    )}  
</ConfirmDialog>  
)  
}
```

V. Страници за създаване и редакция

Създайте компоненти за визуализация на форма, чрез която ще се попълват данните за един паспорт и съответно ще се създава или редактира съществуващ такъв. Може да създадете два отделни компонента, или един единствен.

В репото с кода от текущото упражнение, тези страници са имплементирани, чрез един единствен компонент за форма, като този компонент опционално приема обект Паспорт и използва неговите данни, за да попълни формата предварително. Компонента приема и функцията „onSubmit“, като по този начин за двете страници могат да се имплементират поотделно логиката по изпращане на данните от формата към двата различни ендпойнта.

Чрез използването на един единствен компонент, се намалят шансовете да се забрави да се добави някое ново поле към формата за създаване или редактиране. В някои ситуации обаче, двете форми могат да имат прекалено големи разминавания една от друга, като в тези случаи е по-добре да се използват два отделни компонента.

GitHub репо с кода от текущото упражнение, може да намерите на: <https://github.com/mish0501/uni-practice/tree/lab9>